

Autoencoderi

Nikola Mamić, Petar Matejčić i Jakov Tušek

Lipanj 2026.

1 Uvod

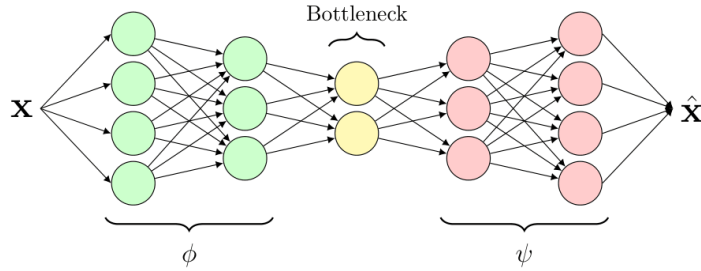
Autoenkoderi su vrsta neuronskih mreža čija je svrha učenje reprezentacija podataka, obično s ciljem redukcije dimenzije ili određivanja značajki. Njihov osnovni cilj jest što točnije reproducirati podatke nakon kodiranja i dekodiranja kroz sloj uskog grla (*bottleneck*). Omogućuju redukciju dimenzionalnosti podataka nenadgledanim učenjem, ali i rekonstrukciju iz reduciranih podataka. Uz redukciju dimenzionalnosti, primjene uključuju detekciju anomalija, uklanjanje šuma iz podataka itd.

U ovom radu predstavljamo osnovnu inačicu modela autoenkodera i nekoliko proširenja, njihove implementacije i primjene na konkretne probleme.

2 Osnovna arhitektura autoenkodera

Pretpostavimo da imamo podatke $\mathbf{x} \in \mathbb{R}^d$. Autoenkoderi se sastoje od dva dijela: enkodera i dekodera. Enkoder je funkcija $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$, pri čemu je tipično $p < d$, tj. latentni prostor ima manju dimenziju od originalnog. Dekoder je funkcija: $\psi : \mathbb{R}^p \rightarrow \mathbb{R}^d$. Enkoder preslikava podatke u latentni prostor, dok dekoder pokušava rekonstruirati originalne podatke. Na slici 1 možemo se vidjeti dijagram autoenkodera. Uočimo, autoenkoder možemo pomoću ϕ i ψ zapisati kao:

$$\hat{\mathbf{x}} = \psi(\phi(\mathbf{x}))$$



Slika 1: Dijagram autoenkodera

Sloj uskog grla je ključno svojstvo autoenkodera, jer upravo taj sloj tipično smanjuje dimenziju podataka, čime tjera neuronsku mrežu da pomno bira važne dijelove podataka i ignorira šum.

Za funkciju gubitka najčešće se uzima srednjekvadratna pogreška (MSE), čime se računa greška između originalnog i rekonstruiranog podatka. Za binarne podatke $\mathbf{x}$ koristi se *cross-entropy* funkcija gubitka, odnosno log-vjerodostojnost podataka $\mathbf{x}$ uz parametre Bernoullijeve distribucije $\hat{\mathbf{x}}$.

$$L(\phi, \psi) = \frac{1}{n} \sum_{i=1}^n \|x^{(i)} - \psi(\phi(x^{(i)}))\|^2$$

$$L(\phi, \psi) = -\frac{1}{n} \sum_{i=1}^n [x^{(i)} \log(\psi(\phi(x^{(i)}))) + (1 - x^{(i)}) \log(1 - \psi(\phi(x^{(i)})))]$$

3 Sparse Autoencoders

Varijanta modela autoenkodera opisana u ovom odjeljku, Sparse Autoencoder (SAE), teži povećanju rijetkosti naučene reprezentacije podataka u sloju uskog grla mreže (nadalje *bottleneck*). Za neuron smatramo da je "aktivan" ako mu je izlazna vrijednost velika, a "neaktivan" ako je ona blizu nule. U tom smislu, da bismo reprezentaciju u *bottleneck* sloju učinili rijetkom, moramo podesiti postupak učenja da penalizira reprezentacije koje nisu rijetke. Za skup podataka na kojima učimo algoritam $\{x^{(1)}, \dots, x^{(N)}\}$, gledamo

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^m [a_j^{(R)}(x^{(i)})],$$

prosječna izlazna (aktivacijska) vrijednost j -tog neurona *bottleneck* sloja, po svim ulaznim podacima.

Želimo učenjem postići

$$\hat{\rho} = \rho,$$

gdje je ρ **parametar rijetkosti**, obično mala vrijednost blizu nule (npr. $\rho = 0.05$ znači da želimo da je prosječna aktivacijska vrijednost neurona u *bottleneck* sloju blizu 0.05).

Kako bi to postigli, dodajemo dodatni penalizacijski sumand našoj funkciji gubitka koja vraća veliku vrijednost ("kažnjava") neurone kojima je prosječna aktivacijska vrijednost $\hat{\rho}_j$ udaljena od ρ :

$$\sum_{j=1}^{R_k} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j},$$

gdje je R_k broj neurona u *bottleneck* sloju. Riječ je zapravo o zbroju Kullback-Leibler (KL) divergencija $\sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j)$, jer tretiramo aktivacijsku vrijednost svakog neurona *bottleneck* sloja kao Bernoullijevu slučajnu varijablu s empirijskim očekivanjem $\hat{\rho}_j$, a naš idealni neuron je Bernoullijeva s očekivanjem ρ . Stoga je naša nova funkcija gubitka oblika

$$C_{\text{sparse}}(W, b) = C(W, b) + \beta \sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j),$$

gdje je $C(W, b)$ funkcija gubitka osnovnog modela (u ovom slučaju MSE), a β hiperparametar koji predstavlja koliko je rijetkost reprezentacije bitna - za $\beta = 0$ model je jednak osnovnom modelu (nema penalizacije gustih reprezentacija).

Pošto penaliziramo samo *bottleneck* sloj, pri *backpropagation*-u jedina je promjena od osnovnog modela pri računanju parcijalne derivacije funkcije gubitka

s obzirom na vrijednost tog sloja prije aktivacije ($\delta_j^{(l)} = \frac{\partial C}{\partial z_j^{(l)}}$, gdje je $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$) - umjesto računanja

$$\delta_i^{(R)} = \left(\sum_{j=1}^{R_k} W_{ji}^{(R)} \delta_j^{(R+1)} \right) f'(z_i^{(R)}),$$

računamo

$$\delta_i^{(R)} = \left(\left(\sum_{j=1}^{R_k} W_{ji}^{(R)} \delta_j^{(R+1)} \right) + \beta \left(-\frac{\rho}{\hat{\rho}_i} + \frac{1-\rho}{1-\hat{\rho}_i} \right) \right) f'(z_i^{(R)}).$$

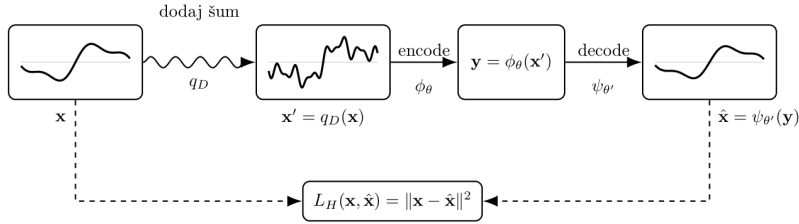
Zbog toga što koristimo $\hat{\rho}$ za ažuriranje parametara u *backpropagation* koraku, prije nego u njega uđemo, moramo napraviti evaluaciju-*forward propagation* korak na svim podacima, kako bismo mogli uprosječiti aktivacijske vrijednosti *bottleneck* sloja na svim podacima. To znači da ćemo u svakoj epohi treniranja mreže dva puta proći po svim podacima (zapravo ovim pristupom, za svako ažuriranje unutar epohe koristimo isti $\hat{\rho}$ kao i na početku te epohe, koji nije aktualan nakon prve promjene parametara mreže; da bismo uvijek koristili aktualni $\hat{\rho}$, morali bi nakon svakog *backpropagation*-a ponovno izračunati novi $\hat{\rho}$, sad drukčiji jer smo promijenili parametre mreže. Naravno, ovo nije moguće u praksi jer bi za n podataka u skupu za treniranje zahtijevalo ukupno $\mathcal{O}(n^2)$ evaluacija/*forward propagation*-a).

4 Denoising Autoencoders

Autoenkoderi se mogu, zahvaljujući sloju uskog grla, koristiti za uklanjanje šuma u podacima. Kao što smo ranije naveli, smanjenje dimenzije u *bottleneck* sloju zahtijeva od autoenkodera da bira najvažnije značajke u podacima, pa se možemo nadati da će prilikom kodiranja šum nestati, a rekonstruirani podaci će biti slični stvarnim "čistim" podacima.

Ključna ideja je to da prilikom treniranja u originalne podatke namjerno dodajemo šum. Tada će neuronska mreža nastojati reproducirati originalne ("čiste") podatke koristeći podatke sa šumom, prilikom čega će naučiti uklanjati šum.

Na slici 2 može se vidjeti prikaz procesa treniranja autoenkodera za otklanjanje šuma. Originalan podatak $\mathbf{x}$ korumpiramo djelovanjem funkcije $q_D(\mathbf{x}) =: \mathbf{x}'$. Potom neuronska mreža kodira korumpirani podatak $\mathbf{y} = \phi_\theta(\mathbf{x}')$. Dekodiranjem $\hat{\mathbf{x}} = \psi_{\theta'}(\mathbf{y})$ dobijemo rekonstrukciju od $\mathbf{x}$, koju uspoređujemo s originalnim ("čistim") podatkom koristeći funkciju gubitka L_H .

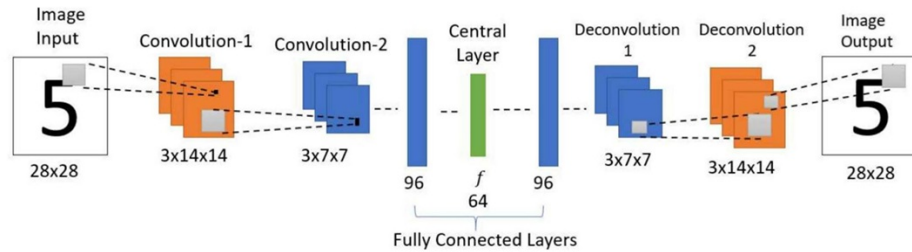


Slika 2: Dijagram autoenkodera za otklanjanje šuma.

Funkcija q_D predstavlja korumpiranje podataka koje može biti vrlo raznoliko: Gaussovski šum, nedostajanje nekih značajki u podacima, dodavanje crnih i bijelih piksela u slikama, itd..

5 Convolutional Autoencoders

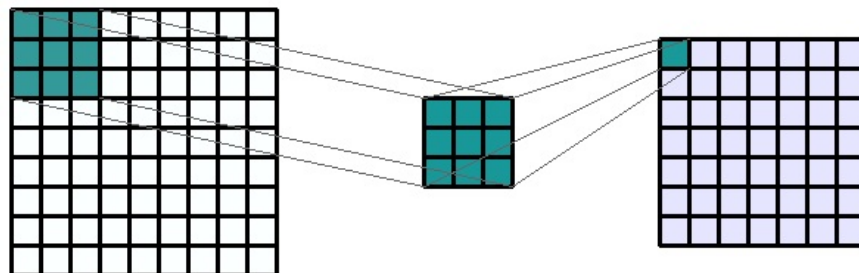
Za slike se obično koriste autoenkoderi s konvolucijskim slojevima. U njima se prvo rade konvolucije, primjenjuje se ReLU aktivacijska funkcija te se dobiveni feature map potom sažima pomoću max poolinga. Nakon određenog broja takvih slojeva slijedi jedan do dva klasična, gusta sloja u kojima su svi neuroni međusobno povezani sa sljedećim slojem. Njihova uloga je da se slika može uložiti u proizvoljnu dimenziju jer sažimanje pomoću max poolinga uvijek smanji dimenziju 4 puta (u svakom kanalu ako ih koristimo više). Nakon toga slijedi dekoder koji ima ponovno simetrične guste slojeve i ponovno podiže dimenziju pomoću slojeva transponiranih konvolucija. Tu arhitekturu možemo vidjeti na sljedećoj slici. Nakon nje slijedi detaljniji opis svakog sloja. Napomenimo da je vrlo važno da se matrica normalizira prije treniranja.



Slika 3: Arhitektura konvolucijskog autoenkodera

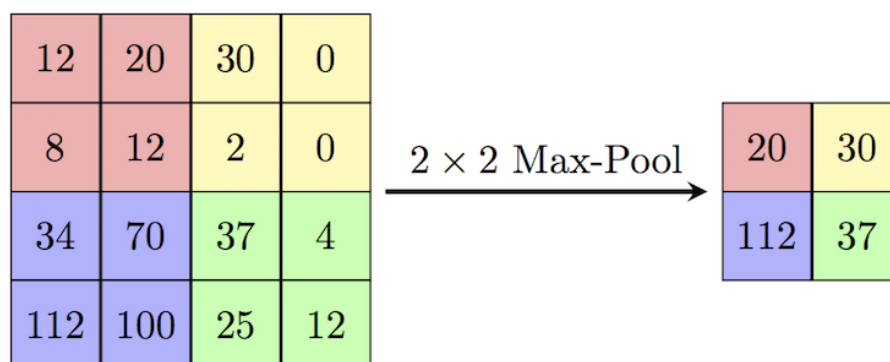
5.1 Konvolucijski slojevi

U ovim slojevima obično koristimo 3x3 linearne filtre. Kako bi feature map koji nastaje primjenom filtra bio iste dimenzije kao inicijalna slika, tj. kako rubovi ne bi bili korišteni manje puta nego centar slike, možemo koristiti padding, tj. dodati nule na rub slike. Feature map se konstruira tako da 3x3 linearni filter klizi preko slike na sve moguće pozicije i pripadna konvolucija se zapisuje u feature map, što je prikazano na Slici 4.



Slika 4: Konvolucija

Nakon toga se na dobiveni feature map primjenjuje ReLU aktivacijska funkcija dana s $f(x) = \max(0, x)$, a potom 2×2 max pooling, što znači da se slika podijeli na 2×2 kvadrate i iz svakog se uzima maksimum, što rezultira sažimanjem slike koristeći samo najvažnije feature. Taj koncept je vizualiziran na Slici 5.



Slika 5: Max pooling

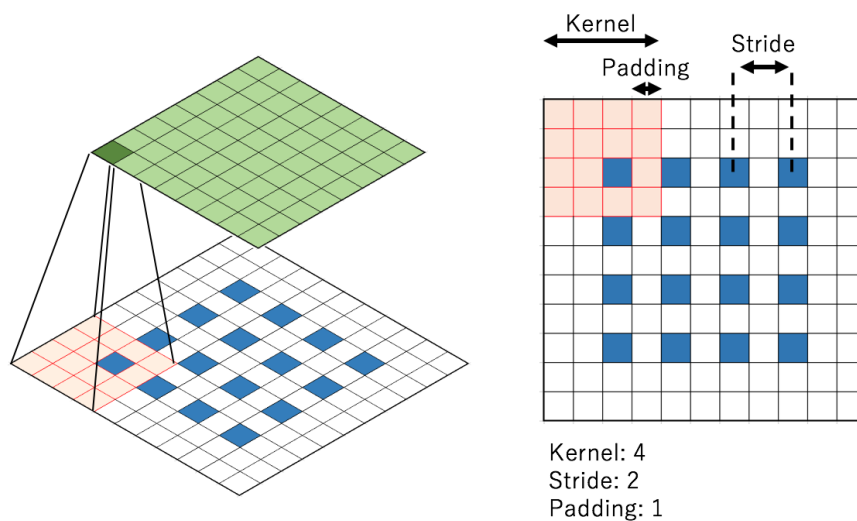
Taj output se dalje propagira idućem sloju.

5.2 Gusti slojevi

Nakon nekoliko konvolucijskih slojeva slika se vektORIZIRA i slijedi osnovni autoencoder opisan u prethodnim poglavljima. Njegov bottleneck je upravo nižedimenzionalna reprezentacija koju želimo.

5.3 Transponirane konvolucije

Dekoder se dalje nastavlja s transponiranim konvolucijama koje ponovno podižu dimenziju slike. One se rade tako da se matrica iz prethodnog sloja popuni nul-redcima i nul-stupcima (svaki drugi redak je nul i svaki drugi stupac) i potom se rade 3×3 konvolucije i primjenjuje ReLU aktivacijska funkcija. Na taj način mreža sama uči kako povećati sliku (mi inicijaliziramo s 0, a u procesu učenja se odredi što zapravo treba napraviti). Zadnji sloj koristi sigmoidnu funkciju da opet dobijemo normaliziranu matricu koja predstavlja našu sliku.



Slika 6: Transponirana konvolucija

6 Primjeri

U ovom odjeljku ćemo demonstrirati primjene prethodno navedenih vrsta autoenkodera. Prikazat ćemo rezultate jednostavnog autoenkodera za otklanjanje šuma na primjeru audio zapisa, korištenje sparse autoenkodera za detekciju anomalija u mrežnom prometa, te primjenu konvolucijskih mreža na slikama ručno pisanih znamenki.

6.1 Uklanjanje šuma iz zvuka

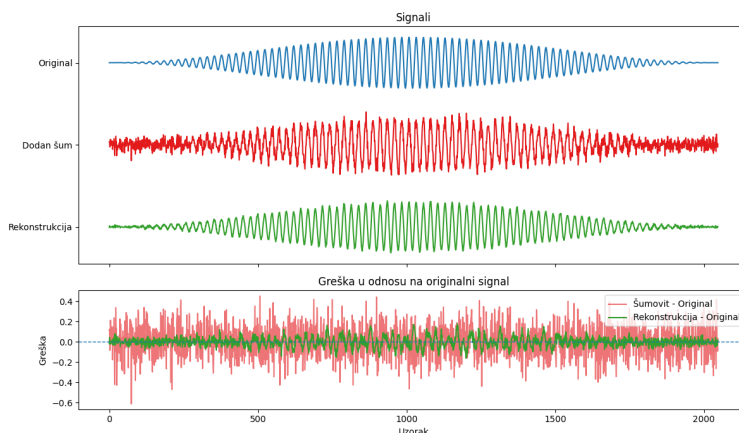
Za početak ćemo prikazati jednostavan autoenkoder koji radi direktno na audio zapisu. Ovaj pristup generalno ne daje dobre rezultate jer neuronska mreža primijenjena na zvuku kao vektoru ne može iskoristiti vremensku zavisnost zvuka. Stoga smo, za potrebe demonstracije autoenkodera, skup podataka smanjili na kombinacije 5 unaprijed određenih nota.

Također, zbog jednostavnosti smo uzeli da je ulazni zvuk uzorkovan 2048 puta, tj. da je on vektor $x \in \mathbb{R}^{2048}$, te da skriveni slojevi preslikavaju x u slijedeće dimenzije:

$$2048 \rightarrow 256 \rightarrow 64 \rightarrow 256 \rightarrow 2048.$$

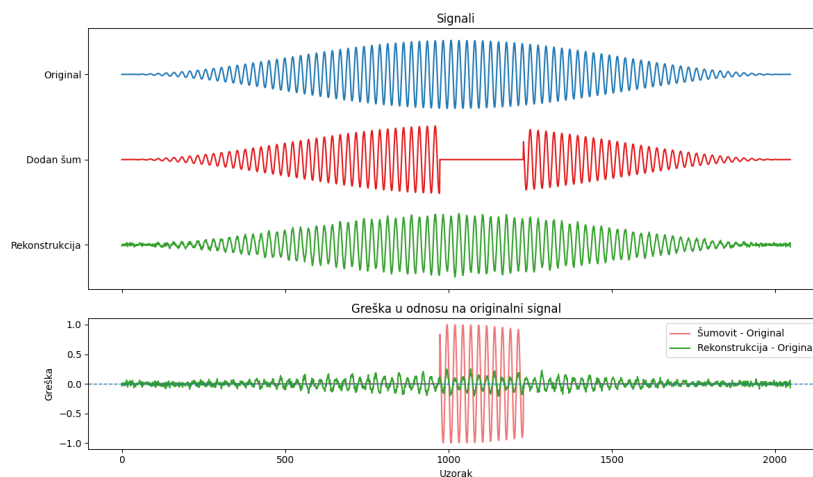
Ovaj model sadrži približno 1,08 milijuna parametara, što uz pomoć SGD-a možemo izračunati u razumnom vremenu. Kao aktivacijsku funkciju smo koristili $\tanh$.

Na slici 7 vidimo rezultat primjene navedenog autoenkodera na zvuk s Gaussovskim šumom. Na donjem grafu vidimo kako se greška rekonstruiranog zvuka smanjila u odnosu na dani šumoviti signal.



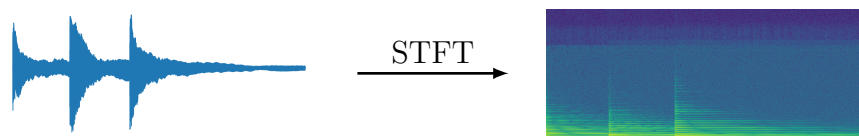
Slika 7: Primjena autoenkodera na uklanjanje Gaussovskog šuma

Na slici 8 prikazan je rezultat primjene autoenkodera treniranog na podacima gdje je jedan dio zvuka uklonjen. Kao što se može vidjeti, neuronska mreža izuzetno točno nadopunjuje zvuk. No, moramo napomenuti da su ovako dobri rezultati posljedica malog skupa podataka te se ovaj pristup ne može dobro generalizirati na općenite zvukove, čak ni na općenite čiste frekvencije.



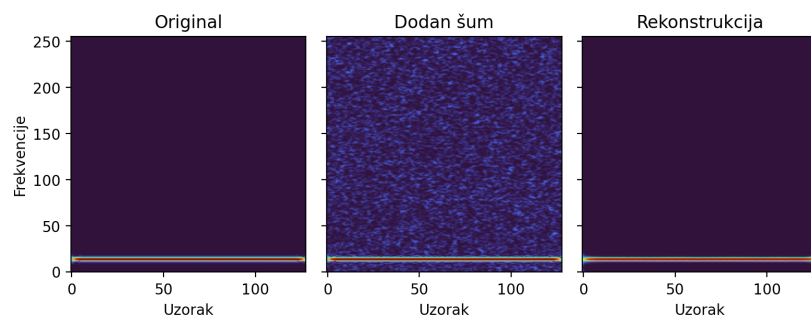
Slika 8: Primjena autoenkodera na nepotpuni zvučni signal

Kako bi mogli ukloniti šum iz kompliciranijih i dužih audio zapisa, moramo nekako uzeti u obzir vremensku strukturu zvuka. To radimo uz pomoć spektrograma dobivenih STFT algoritmom. Tako na primjer možemo jednu sekundu zvuka, uzorkovanu 16,000 puta u sekundi, svesti na 256×128 matricu spektrograma:

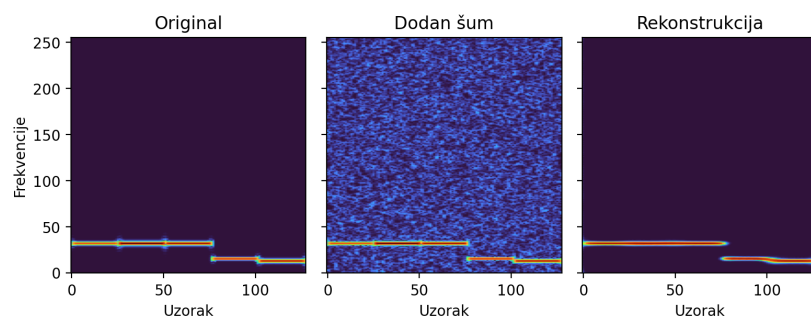


Time smo uklanjanje šuma iz audio zapisa sveli na uklanjanje šuma iz spektrograma (slike). Stoga, možemo koristiti konvolucijske slojeve kako bi dodatno saželi spektrogram i smanjili broj potrebnih parametara u modelu.

Na slikama 9 i 10 vidimo originalne spektrograme, spektrograme audio zapisa s dodanim šumom te rekonstruirane spektrograme za dva jednostavna primjera.

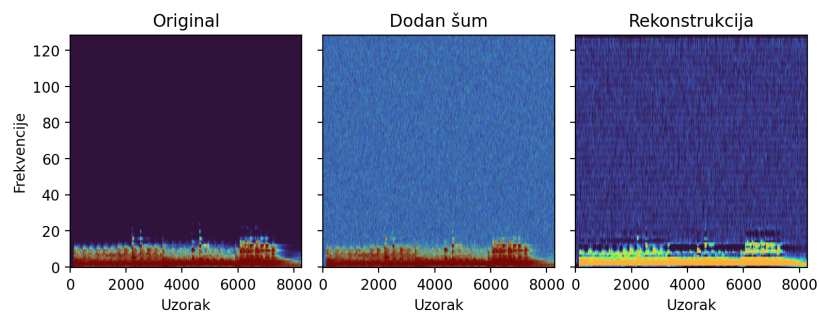


Slika 9: Audio zapis s jednom notom



Slika 10: Audio zapis s četiri note

Konačno, na slici 11 možemo vidjeti kako autoenkoder djeluje na spektrogram kompliciranijeg audio zapisa. Uočimo kako je šum u rekonstrukciji tamnije boje, tj. prigušene su frekvencije koje pripadaju šumu.



Slika 11: Uklanjanje šuma u kompliciranijem audio zapisu

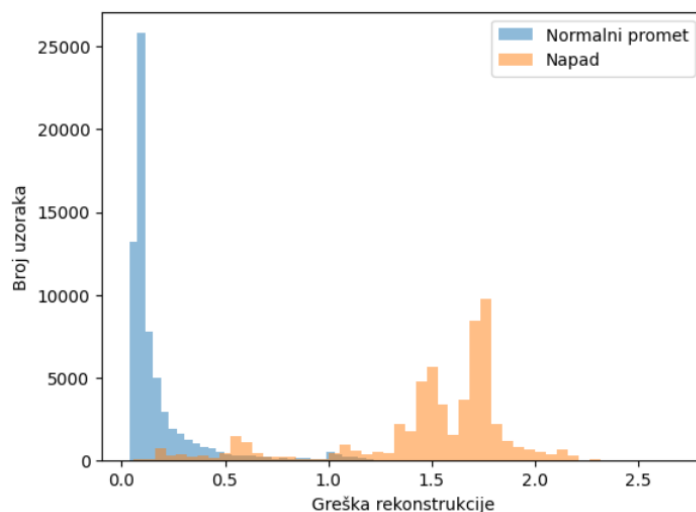
6.2 Detekcija anomalija u mrežnom prometu

Jedan od najpoznatijih skupova podataka u području kibernetičke sigurnosti je KDD99 (Knowledge Discovery and Data Mining Tools Competition 99) - skup zapisa TCP veza simulirane vojne mreže koju su postavili DARPA i MIT 1998. kroz devet tjedana. Dio zapisa odgovara standardnom prometu, ali neki su zapisi pomno izrađeni kibernetički napadi. Cilj je bio napraviti standardizirani *benchmark* za usporedbu sustava za detekciju anomalija u mrežnom prometu. NSL-KDD je modernija i "očišćena" inačica tog skupa podataka.

Skup se sastoji od više od 120 000 zapisa, od kojih svaki ima 125 atributa. Otprilike pola zapisa predstavlja normalan mrežni promet, a ostala polovica mrežne napade.

Ideja je trenirati autoencoder isključivo na standardnim zapisima. Tada je očekivano da će pri rekonstrukciji standardnih zapisa greška biti mala, a pri rekonstrukciji zapisa napada greška biti veća. Penaliziranjem rješenja koja nisu rijetka tjeramo mrežu da centralizira aktivacije u nekolicinu neurona. U suštini, ovdje je problem suprotan od problema za koji je dobar model Denoising Autoencoder-a, koji uklanja šum u podacima. Pokušavamo što više naglasiti razlike u podacima kako bismo odvojili napade od normalnog prometa. Naravno, sofisticiraniji napadi koji bolje oponašaju normalan promet bit će teži za detektirati od naivnijih malicioznih zapisa.

Uz mrežu dubine 3 sa slojevima veličine 122, 32, 122, $\beta = 0.5$, $\rho = 0.2$ i stopom učenja $\eta = 0.01$ te 3 epohe, dobiveni su sljedeći rezultati:



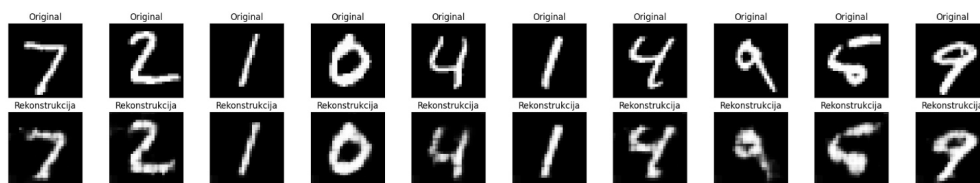
Slika 12: Histogram grešaka rekonstrukcije zapisa mrežnog prometa

Klasificiramo zapis kao napad ako je greška rekonstrukcije iznad praga, npr. ovdje 0.5. Ukupno je 2414 napada koji su ispod praga, tj. koje ovom metodom ne bismo uspješno klasificirali kao napade. Ako pogledamo o kojim je napadima riječ, vidimo da je, kao i očekivano, riječ o napadima koje je teško raspoznati iz samih TCP paketa, jer je maliciozni sadržaj van konteksta samog zapisa (ili je riječ o malicioznom sadržaju koji se pošalje preko uspostavljene legitimne TCP veze, kao u slučaju *warezmaster* napada, ili je riječ o valjanim zapisima koji uzrokuju rušenje sustava ili slične komplikacije na aplikacijskom sloju, kao što je to kod *back* napada, koji uzrokuje *denial of service* na Apache web serverima). Kod napada gdje je jasno vidljiva patološka struktura zapisa, kao kod *neptune* napada, koji šalje velik broj kratkotrajnih zahtjeva bez slanja podataka ("zatrpara" poslužitelj beskorisnim zahtjevima kako bi mu popunio tablicu veza i onemogućio spajanje legitimnih zahtjeva) - naš model uspješno klasificira svih 41 214 zapisa kao napade!

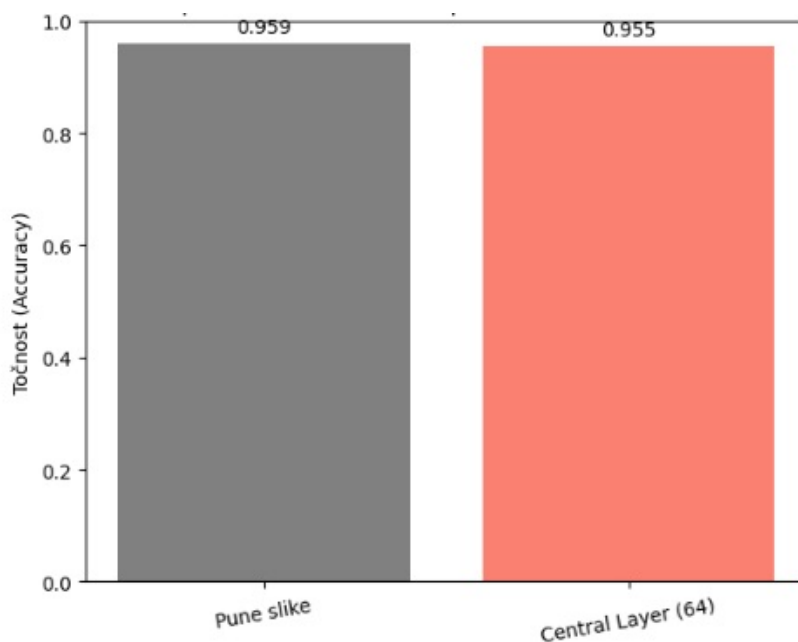
Tip napada	Propušteno	Ukupno	% propuštenih	Avg. greška
warezmaster	19	20	95.0%	0.275
warezclient	762	890	85.6%	0.329
back	784	956	82.0%	0.320
multihop	5	7	71.4%	0.595
ftp_write	4	8	50.0%	0.591
rootkit	4	10	40.0%	0.634
buffer_overflow	6	30	20.0%	0.901
nmap	289	1493	19.4%	0.759
loadmodule	1	9	11.1%	0.889
ipsweep	303	3599	8.4%	0.675
satan	237	3633	6.5%	1.599
guess_passwd	0	53	0.0%	1.452
imap	0	11	0.0%	1.592
land	0	18	0.0%	1.919
neptune	0	41214	0.0%	1.683
perl	0	3	0.0%	1.236
phf	0	4	0.0%	1.037
pod	0	201	0.0%	1.190
portsweep	0	2931	0.0%	1.821
smurf	0	2646	0.0%	1.445
spy	0	2	0.0%	0.728
teardrop	0	892	0.0%	1.262

6.3 MNIST

Konvolucijski autoenkoder smo isprobali na MNIST-u. Glavna mreža je imala točno arhitekturu prikazanu u četvrtom poglavlju. Promatrana je kvaliteta rekonstrukcije slika i koliko dobro SVM može naučiti razliku između znamenki na temelju cijele slike i komprimirane. Implementacija je izvedena pomoću Pythonove Keras biblioteke. Korišteno je 15 epoha učenja. Ovaj autoenkoder reducira dimenziju naših slika sa 784 na 64.



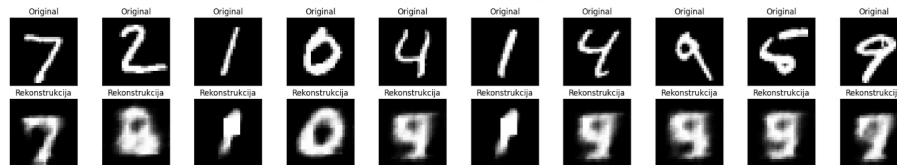
Slika 13: Originalne vs. rekonstruirane slike na test podacima



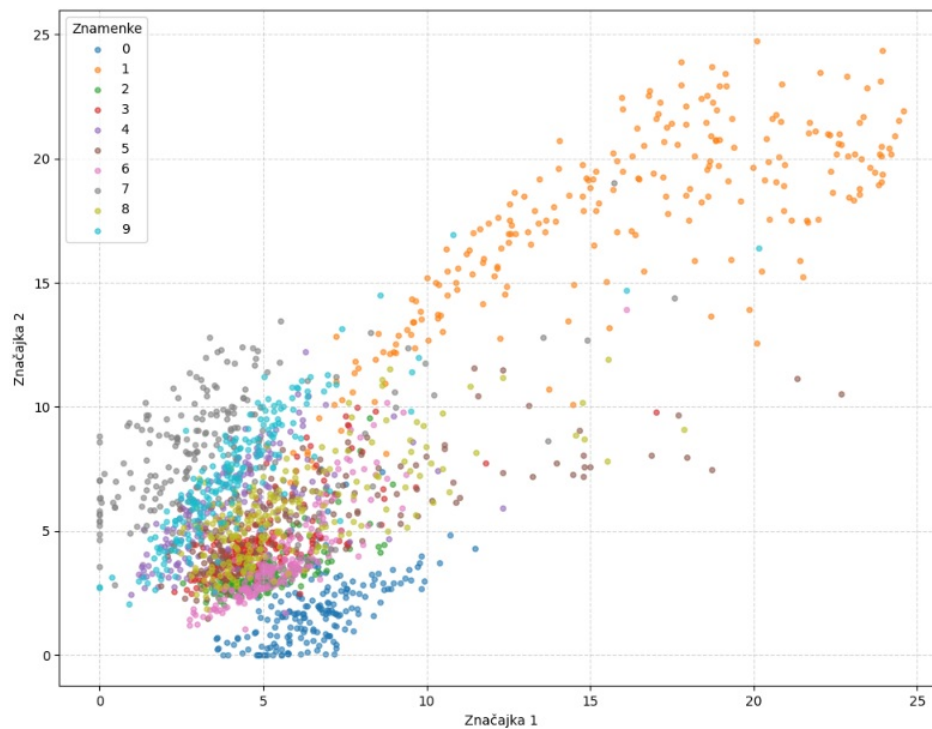
Slika 14: Točnost klasifikacije SVM algoritma na punim i reduciranim slikama

Vidimo da rekonstrukcije vizualno dobro izgledaju te SVM algoritam podjednako dobro može učiti razliku između znamenki i na reduciranim slikama što znači da je to ulaganje dosta dobro.

Potom smo, vizualizacije radi, napravili ulaganje u dvodimenzionalan prostor. Jasnije je da je to premala dimenzija da bi rekonstrukcije dobro izgledale ali je zanimljivo da se u ravnini dosta jasno razaznaju klasteri.



Slika 15: Originalne vs. rekonstruirane slike na test podacima za dvodimenzional bottleneck



Slika 16: Ulaganje u 2D prostor test podataka

7 Zaključak

U ovom smo radu demonstrirali tri inačice autoekodera.

Vidimo da općenita ideja autoenkodera omogućuje primjenu na širokom spektru problema, od potiskivanja šuma u audio zapisima do isticanja anomalija u podacima o mrežnom prometu. Na MNSIT podacima prikazali smo kako autoenkoderi mogu služiti za redukciju dimenzije podataka bez velikog gubitka u mogućnosti raspoznavanja.

Za kraj napomenimo kako postoje još brojne inačice autoenkodera s raznolikim primjenama.

Literatura

- [1] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [2] Ibomoiye Domor Mienye and Theo G. Swart. Deep autoencoder neural networks: A comprehensive review and new perspectives. *Archives of Computational Methods in Engineering*, 32:3981–4000, 2025.
- [3] Andrew Ng. Sparse autoencoder. CS294A Lecture Notes, Stanford University, 2011.
- [4] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. ICML '08, page 1096–1103, New York, NY, USA, 2008. Association for Computing Machinery.